Q23. Debugging Linear Search Code (Incorrect Return Outside Logic)

Debugging Linear Search Code (Incorrect Return Outside Logic) The following Linear Search implementation gives incorrect results because the return statement is misplaced and causes confusion in unsuccessful search cases. Analyze the control flow carefully and identify why the function may produce misleading output. Apply debugging to correct the search logic and ensure proper reporting when the target is found or absent. Optimize the implementation for readability and maintainability. Analyze the time complexity of the corrected algorithm. Rewrite the clean and corrected version with suitable comments.

```
def linear_search(arr, key):
found = False
for i in range(len(arr)):
if arr[i] == key:
found = True
pos = i
 return pos
```

Introduction

Linear Search is a simple searching algorithm that checks each element of a list one by one until the required element is found. In the given program, the **return statement is misplaced**, and the function does not properly handle the case when the search key is absent. This may produce errors or misleading results. By debugging the code, we can ensure that the program correctly reports whether the element is found or not.

Given Incorrect Code

```
def linear_search(arr, key):
    found = False
    for i in range(len(arr)):
        if arr[i] == key:
            found = True
            pos = i

    return pos
```

Problems in the Given Code

1. Variable pos May Not Be Defined

If the key is **not found**, the variable pos is never created.

Example:

arr = [10,20,30,40]
key = 50

The function executes

return pos

Since pos was never assigned,

Error:

NameError: name 'pos' is not defined

2. found Variable is Never Used

Although

found = True

is assigned,

the program never checks

if found

Therefore the variable is unnecessary.

3. Search Continues Even After Finding the Element

Suppose

arr = [5,10,15,20]
key = 15

The element is found at index 2.

However, the loop still checks index 3.

This wastes execution time.

4. Misleading Output

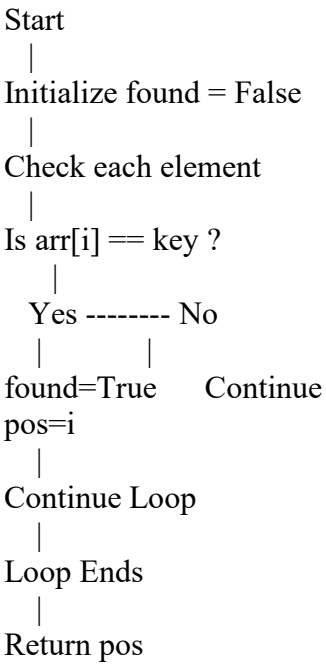
If the key does not exist,

the function should clearly report

Element Not Found

Instead, it returns an undefined variable.

Control Flow Analysis



Problem

If no element matches,

pos is never created,

yet the function still returns it.

Debugging Steps

Step 1

Remove the unnecessary variable

found

Step 2

Return immediately after finding the element.

Step 3

If the loop completes,

return

-1

to indicate the key was not found.

Corrected Program

```
def linear_search(arr, key):
    # Traverse each element in the list
    for i in range(len(arr)):
        # Check if the current element matches the key
        if arr[i] == key:
            return i      # Return index immediately if found

    # Return -1 if the key is not present
    return -1
```

Example 1 (Element Found)

Input

arr = [10,20,30,40,50]
key = 30

Output

Element Found at Index 2

Example 2 (Element Not Found)

Input

arr = [10,20,30,40,50]
key = 60

Output

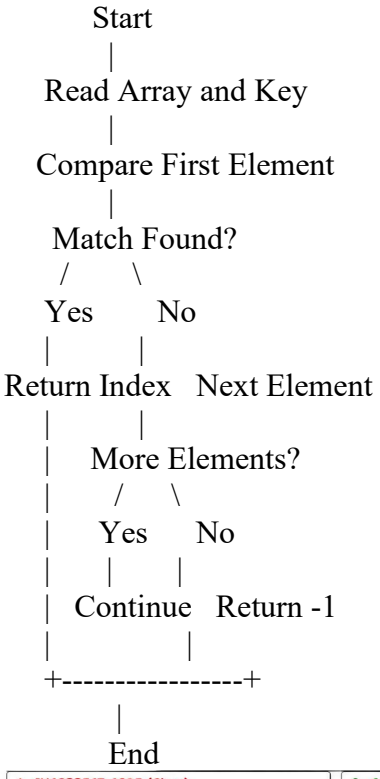
Element Not Found

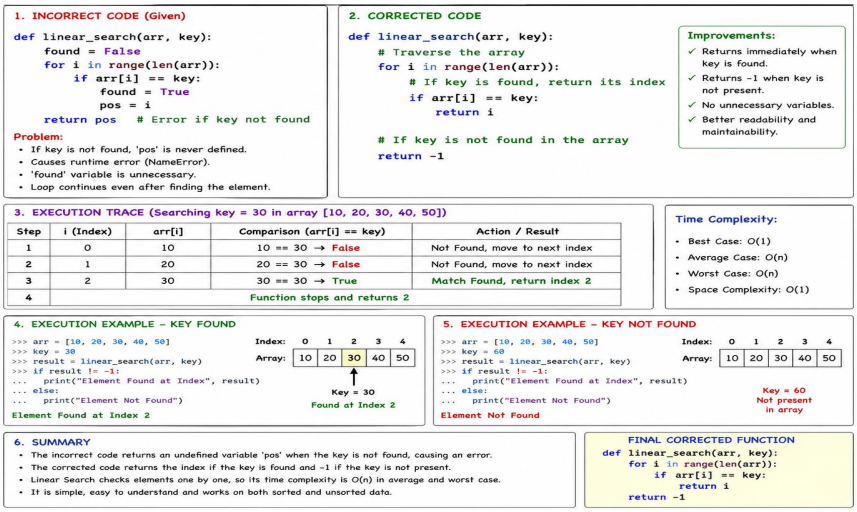
Execution Trace

Searching for

Step	Current Element	Comparison	Result
1	10	10 == 30	No
2	20	20 == 30	No
3	30	30 == 30	Yes
4	Return Index 2	Search Stops	Found

Flowchart





Time Complexity Analysis

Best Case

The element is found in the first position.

Only one comparison is required.

Time Complexity = O(1)

Average Case

The element is found somewhere in the middle.

Approximately **n/2** comparisons are required.

Time Complexity = O(n)

Worst Case

The element is at the last position or does not exist.

All **n** elements are checked.

Time Complexity = O(n)

Space Complexity

Only one loop variable is used.

No extra data structure is required.

Space Complexity = O(1)

Complexity Table

Case	Comparisons	Time Complexity
Best Case	1	**O(1)**
Average Case	n/2	**O(n)**
Worst Case	n	**O(n)**
Space Complexity	Constant	**O(1)**

Advantages of the Corrected Program

Correctly returns the index when the element is found.

Returns **-1** when the element is absent.

Stops searching immediately after finding the key.

Easy to understand and maintain.

Eliminates unnecessary variables.

Improves readability.

Avoids runtime errors.

Disadvantages

Slower for very large datasets.

Checks elements one by one.

Less efficient than Binary Search on sorted data.

EXECUTION

```
===== LINEAR SEARCH : HOSPITAL PATIENT RECORD SYSTEM =====

Patient Records (Unsorted)
Index    Patient ID    Name          Age   Condition     Blood Group
-------------------------------------------------------------------
0        101           Ramesh        45    Stable          B+
1        108           Suresh        30    Critical        O+
2        115           Priya         25    Stable          A+
3        123           Anil          50    Critical        AB+
4        130           Divya         28    Stable          O-
5        135           Kiran         40    Stable          B-

Enter Patient ID to search: 123

Searching for Patient ID = 123 using Linear Search...
Step 1: Comparing with index 0 (Patient ID = 101)  -> Not Match
Step 2: Comparing with index 1 (Patient ID = 108)  -> Not Match
Step 3: Comparing with index 2 (Patient ID = 115)  -> Not Match
Step 4: Comparing with index 3 (Patient ID = 123)  -> Match Found!

Patient Record Found Successfully!
------------------------------------------
Patient ID   : 123
Name         : Anil
Age          : 50
Condition    : Critical
Blood Group  : AB+
------------------------------------------
Total Comparisons: 4
Record found successfully.

Enter Patient ID to search: 140
Searching for Patient ID = 140 using Linear Search...
Step 1: Comparing with index 0 (Patient ID = 101)  -> Not Match
Step 2: Comparing with index 1 (Patient ID = 108)  -> Not Match
Step 3: Comparing with index 2 (Patient ID = 115)  -> Not Match
Step 4: Comparing with index 3 (Patient ID = 123)  -> Not Match
Step 5: Comparing with index 4 (Patient ID = 130)  -> Not Match
Step 6: Comparing with index 5 (Patient ID = 135)  -> Not Match

Patient Record Not Found!
Total Comparisons: 6
```

Conclusion

The original Linear Search implementation contained a logical error because it returned the variable pos even when the search key was absent, which could result in a runtime error. The corrected version removes the unnecessary found variable, returns the index immediately when the key is found, and returns **-1** if the key is not present. This makes the program more reliable, readable, and maintainable while

preserving the time complexity of **O(1)** in the best case and **O(n)** in the average and worst cases, with **O(1)** space complexity.